Unicesumar

Engenharia de Software

RELATÓRIO FINAL — ENTREGA DO 2º BIMESTRE

INTEGRAPSE — GERENCIADOR DE AÇÕES DO PSE
(Programa Saúde na Escola)

Atividade Extensionista Prática (AEP) — Segunda etapa: implementação, revisão dos artefatos, testes e apresentação

Alunos(a): Isadora Miura – RA: 26002751-2
João Victor Leal Castro – RA: 26010665-2
Pedro Henrique Bergamini Sakamoto – RA: 26009883-2

Maringá - Paraná
2026

SUMÁRIO

1 Introdução
- 1.1 Finalidade deste relatório
- 1.2 Contexto
- 1.3 Limites e cuidados

2 Visão geral do software
- 2.1 Funcionalidades
- 2.2 Atendimento aos requisitos

3 Requisitos e compilação
- 3.1 Requisitos
- 3.2 Instalação do compilador
- 3.3 Compilação
- 3.4 Execução

4 Estrutura do programa
- 4.1 Organização do código
- 4.2 Estruturas de dados
- 4.3 Funções
- 4.4 Fluxo de execução

5 Utilização do programa
- 5.1 Menu principal
- 5.2 Cadastrar ação
- 5.3 Listar ações
- 5.4 Pesquisar ações
- 5.5 Atualizar situação
- 5.6 Resumo geral
- 5.7 Encerrar o programa

6 Validações e mensagens de erro

7 Testes e resultados
- 7.1 Metodologia
- 7.2 Casos de teste
- 7.3 Discussão dos resultados

8 Limitações

9 Melhorias futuras

10 Conclusão da implementação

1 Introdução

1.1 Finalidade deste relatório

Este relatório final documenta o programa Gerenciador de Ações do PSE, desenvolvido em linguagem C como trabalho acadêmico. Seu propósito é permitir que o leitor compreenda o que o software faz e como ele está organizado, compile-o em seu computador, utilize todas as suas funcionalidades e avalie, com base em testes, os resultados obtidos, as limitações existentes e as melhorias possíveis.

1.2 Contexto

O Programa Saúde na Escola (PSE) é uma política que articula as áreas da saúde e da educação para promover a saúde de estudantes da rede pública. Suas atividades são realizadas de forma coletiva nas escolas, por meio de ações como palestras, oficinas e campanhas educativas. Cada ação precisa ser planejada, acompanhada e registrada, para que se saiba quais foram realizadas, quais foram canceladas e quantas pessoas participaram.

O programa descrito neste manual atende a essa necessidade de controle de forma simples, em ambiente de console (terminal), sem recursos avançados, com o objetivo de exercitar condicionais, laços de repetição, vetores, estruturas e funções.

1.3 Limites e cuidados

O sistema foi concebido com os seguintes limites, que também orientam o uso correto do programa:

- **não** cadastra nome, diagnóstico, prontuário, condição clínica ou qualquer dado sensível individual de estudantes;
- **não** realiza diagnóstico, triagem médica, prescrição ou recomendação de tratamento;
- guarda os dados **apenas em memória** durante a execução, sem arquivo, banco de dados, interface gráfica, aplicação web ou bibliotecas avançadas.

2 Visão geral do software

2.1 Funcionalidades

O programa apresenta um menu principal que se repete até que o usuário escolha encerrar. A Tabela 1 resume as opções disponíveis.

Tabela 1 – Opções do menu principal

Opção	Funcionalidade	Descrição
1	Cadastrar ação	Registra código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial.
2	Listar ações	Exibe todas as ações cadastradas em formato de tabela.
3	Pesquisar ações	Localiza ações por código (busca exata) ou por escola ou tema (busca parcial, sem diferenciar maiúsculas de minúsculas).
4	Atualizar situação	Altera a situação para planejada, realizada ou cancelada; quando realizada, registra a quantidade efetiva de participantes.
5	Resumo geral	Mostra a quantidade de ações por situação, o total de participantes e o percentual de participação em relação ao previsto.
0	Sair	Encerra o programa.

Fonte: elaborado pelo autor.

2.2 Atendimento aos requisitos

A Tabela 2 relaciona os requisitos do trabalho à parte do programa que os atende.

Tabela 2 – Requisitos do trabalho e como são atendidos

Requisito	Como é atendido
Cadastrar ação (item 1)	Função `cadastrarAcao`, opção 1 do menu.
Listar ações (item 2)	Função `listarAcoes`, opção 2, com tabela organizada.
Pesquisar por código, escola ou tema (item 3)	Função `pesquisarAcoes`, opção 3.
Atualizar situação e registrar efetivos (item 4)	Função `atualizarSituacao`, opção 4.
Resumo geral com percentual (item 5)	Função `gerarResumo`, opção 5.
Validação de entradas (item 6)	Funções `lerTexto`, `lerInteiro`, `lerData`, `codigoExiste` e `exibirMenu`.
Menu em estrutura de repetição	Laço `do...while` na função `main`.
Condicionais, laços, vetores e funções	Vetor `acoes`, estruturas `if`, `switch`, `for`, `while` e 17 funções.
Funções com responsabilidades claras	Cada função tem uma tarefa; `main` apenas controla o menu.
Mensagens claras	Confirmação de sucesso e explicação de cada erro (Capítulo 6).
Armazenamento em memória	Vetor global de estruturas, sem uso de arquivos.

Fonte: elaborado pelo autor.

2.3 Artefatos revisados e rastreabilidade

Esta seção consolida os artefatos de Engenharia de Software da segunda entrega. Os requisitos e casos de uso foram revisados à luz da proposta da AEP e do código-fonte main.c. O manual, as capturas de tela e os 25 testes registrados nas seções seguintes foram preservados. Os fluxogramas e pseudocódigos da primeira etapa não foram fornecidos para esta revisão e deverão ser anexados e conferidos com a implementação antes da entrega final.

2.4 Requisitos funcionais

Código	Nome	Especificação
RF01	Cadastrar ações	Registrar código, escola, tema, data, público-alvo, responsável, participantes previstos e situação inicial.
RF02	Listar ações	Exibir os registros cadastrados em tabela.
RF03	Pesquisar ações	Pesquisar por código exato ou por escola/tema com correspondência parcial.
RF04	Atualizar situação	Alterar a situação para planejada, realizada ou cancelada conforme as regras de transição.
RF05	Registrar participantes	Informar participantes efetivos quando a ação estiver realizada.
RF06	Gerar resumo	Exibir contagem por situação, total efetivo e percentual de participação.
RF07	Validar entradas	Recusar códigos duplicados, campos vazios, valores inválidos, datas incorretas e opções inexistentes.
RF08	Gerenciar menu	Permitir executar operações repetidamente e encerrar pela opção 0.

2.5 Requisitos não funcionais

Código	Especificação
RNF01	Implementação em linguagem C, compatível com C99.
RNF02	Interface textual em terminal.
RNF03	Armazenamento temporário em vetor de estruturas na memória.
RNF04	Capacidade máxima de 100 ações.

RNF05	Organização modular em funções com responsabilidades distintas.
RNF06	Mensagens claras de orientação, confirmação e erro.
RNF07	Uso exclusivo de dados fictícios e coletivos, sem dados clínicos individuais.
RNF08	Tratamento de entradas inválidas e dos limites de armazenamento.

2.6 Atores e casos de uso

Ator principal: operador da equipe escolar ou de saúde, responsável por registrar e acompanhar ações coletivas do PSE. O sistema não implementa autenticação ou perfis distintos. As validações (RF07) são comportamentos incorporados aos casos de uso, não uma operação independente do usuário.

2.6.1 Catálogo de casos de uso

UC01 — Cadastrar ação

Ator: Operador da equipe escolar/de saúde
Objetivo: Cadastrar ação
Requisitos relacionados: RF01, RF05, RF07
Pré-condições: Menu disponível e capacidade inferior a 100 ações.
Fluxo principal: 1. Selecionar 1; 2. Informar código e campos obrigatórios; 3. Informar participantes previstos e situação; 4. Se realizada, informar efetivos; 5. Sistema valida e confirma cadastro.
Fluxos alternativos: Código repetido, campo inválido, data inválida ou limite atingido: sistema informa erro e cancela a operação.
Pós-condições: Nova ação incluída no vetor; total incrementado.

UC02 — Listar ações

Ator: Operador da equipe escolar/de saúde
Objetivo: Listar ações
Requisitos relacionados: RF02
Pré-condições: Sistema iniciado.
Fluxo principal: 1. Selecionar 2; 2. Sistema percorre os registros e exibe tabela e total.
Fluxos alternativos: Sem registros: informar que nenhuma ação foi cadastrada.
Pós-condições: Registros consultados sem alterações.

UC03 — Pesquisar ações

Ator: Operador da equipe escolar/de saúde
Objetivo: Pesquisar ações
Requisitos relacionados: RF03, RF07
Pré-condições: Sistema iniciado.
Fluxo principal: 1. Selecionar 3; 2. Escolher código, escola ou tema; 3. Informar critério; 4. Sistema apresenta as correspondências.
Fluxos alternativos: Critério inválido: cancelar pesquisa; nenhuma correspondência: apresentar aviso; sem ações: informar ausência de cadastros.
Pós-condições: Resultados exibidos sem alterar registros.

UC04 — Atualizar situação

Ator: Operador da equipe escolar/de saúde
Objetivo: Atualizar situação
Requisitos relacionados: RF04, RF05, RF07

Pré-condições: Existir ação cadastrada com código informado.
Fluxo principal: 1. Selecionar 4; 2. Informar código; 3. Conferir situação atual; 4. Escolher nova situação; 5. Se realizada, informar efetivos; 6. Sistema valida e confirma.
Fluxos alternativos: Código inexistente, entrada inválida ou transição proibida: informar erro e manter situação; nova situação não realizada: zerar efetivos.
Pós-condições: Situação e participantes efetivos atualizados conforme as validações do sistema.

UC05 — Gerar resumo geral

Ator: Operador da equipe escolar/de saúde
Objetivo: Gerar resumo geral
Requisitos relacionados: RF06
Pré-condições: Sistema iniciado.
Fluxo principal: 1. Selecionar 5; 2. Sistema conta ações por situação; 3. Soma efetivos e previstos das realizadas; 4. Calcula e exibe percentual.
Fluxos alternativos: Sem ações realizadas ou soma prevista igual a zero: percentual exibido como 0%.
Pós-condições: Resumo exibido, sem alterar registros.

UC06 — Encerrar programa

Ator: Operador da equipe escolar/de saúde
Objetivo: Encerrar programa
Requisitos relacionados: RF08
Pré-condições: Sistema iniciado.
Fluxo principal: 1. Selecionar 0; 2. Sistema exibe mensagem de saída e termina.
Fluxos alternativos: Opção inválida: sistema informa erro e mantém o menu.
Pós-condições: Programa encerrado; dados temporários deixam de estar disponíveis.

2.8 Matriz de rastreabilidade

Requisito	Função / implementação	Caso de uso
RF01	cadastrarAcao()	UC01
RF02	listarAcoes()	UC02
RF03	pesquisarAcoes()	UC03
RF04	atualizarSituacao()	UC04
RF05	cadastrarAcao(); atualizarSituacao()	UC01; UC04
RF06	gerarResumo()	UC05
RF07	lerTexto(); lerInteiro(); lerData(); codigoExiste(); exibirMenu()	UC01; UC03; UC04
RF08	main(); exibirMenu()	UC01–UC06

2.9 Pendências de integração da primeira etapa

Antes da submissão, anexar e revisar: (a) fluxograma geral do menu; (b) fluxograma detalhado de uma operação; (c) pseudocódigos produzidos na primeira entrega; (d) planejamento das sprints e eventuais alterações de requisitos. O arquivo original deste manual não contém esses artefatos completos. O diagrama gráfico de casos de uso também deverá ser inserido caso seja exigida sua representação visual; as descrições textuais acima não o substituem.

3 Requisitos e compilação

3.1 Requisitos

Para compilar e executar o programa são necessários apenas um compilador C compatível com o padrão C99 (por exemplo, o GCC), um terminal e o arquivo de código-fonte `main.c`. O programa usa somente as bibliotecas padrão `stdio.h`, `stdlib.h`, `string.h` e `ctype.h`, portanto não há nada além do compilador a instalar. Os testes deste manual foram feitos com o **GCC 13.3.0** em Ubuntu 24.04.

3.2 Instalação do compilador

- **Linux (Debian/Ubuntu):** `sudo apt install build-essential`.
- **macOS:** `xcode-select --install`, que instala as ferramentas de linha de comando com o compilador.
- **Windows:** instalar o MinGW-w64 (por exemplo, pelo MSYS2), usar o WSL (Linux dentro do Windows) ou uma IDE que já traga o compilador, como o Code::Blocks.

3.3 Compilação

Abra o terminal na pasta onde está o arquivo `main.c` e execute:

```
gcc -Wall -Wextra -std=c99 main.c -o pse
```

As opções usadas estão descritas na Tabela 3. Se tudo estiver correto, o compilador **não exibe nenhuma mensagem** e cria o executável `pse`. A ausência de avisos é um critério de qualidade do trabalho.

Tabela 3 – Opções de compilação

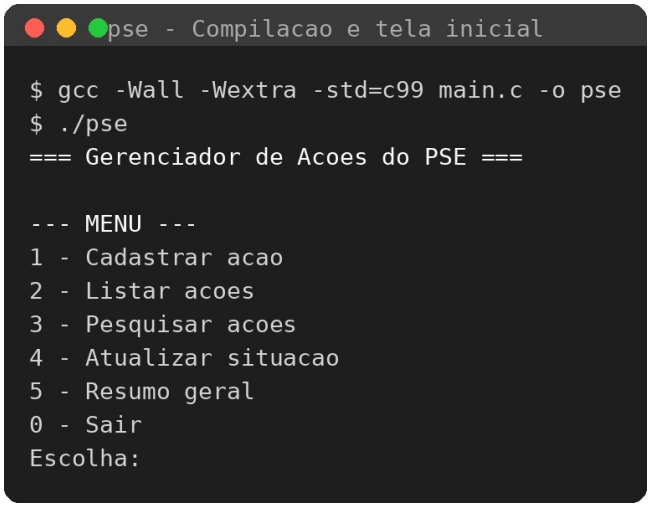
Opção	Função
`-Wall`	Ativa os principais avisos do compilador, que ajudam a encontrar erros comuns.
`-Wextra`	Ativa avisos adicionais, como parâmetros declarados e não utilizados.
`-std=c99`	Exige que o código siga o padrão C99.
`main.c`	Arquivo com o código-fonte.
`-o pse`	Define o nome do programa gerado.

Fonte: elaborado pelo autor.

3.4 Execução

No Linux e no macOS, execute `./pse`. No Windows, o compilador costuma gerar o arquivo `pse.exe`, que é executado digitando `pse` no terminal. A Figura 1 mostra a compilação e a tela inicial com o menu principal.

Figura 1 – Compilação e tela inicial do programa



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

Por compatibilidade com diferentes terminais, o programa exibe seus textos sem acentos (por exemplo, “Acao” e “Saude”) e espera que os dados também sejam digitados **sem acentos e sem cedilha**. O motivo é explicado no Capítulo 8.

4 Estrutura do programa

4.1 Organização do código

Todo o código está em um único arquivo, `main.c`, organizado na seguinte ordem:

1. inclusão das bibliotecas e definição da constante `MAX_ACOES` (100);
2. definição do tipo `Situacao` (enumeração) e da estrutura `Acao`;
3. declaração das variáveis globais `acoes` e `totalAcoes`;
4. protótipos das funções, que permitem ao compilador conhecer as funções antes de elas serem definidas;
5. função `main` e, em seguida, as demais funções.

4.2 Estruturas de dados

A situação de uma ação é representada por uma enumeração, o que evita “números mágicos” espalhados pelo código e deixa a leitura mais clara:

```
typedef enum { PLANEJADA = 1, REALIZADA, CANCELADA } Situacao;
```

A enumeração começa em 1 para que o valor 0 nunca seja confundido com uma situação válida. Cada ação é guardada em uma estrutura `Acao`, descrita na Tabela 4.

Tabela 4 – Campos da estrutura `Acao`

Campo	Tipo	Conteúdo
`codigo`	`int`	Identificador único da ação (1 a 999999).
`escola`	`char[60]`	Nome da escola (até 59 caracteres).
`tema`	`char[60]`	Tema da ação (até 59 caracteres).
`data`	`char[11]`	Data prevista no formato DD/MM/AAAA (10 caracteres e o terminador).
`publicoAlvo`	`char[40]`	Público-alvo (até 39 caracteres).
`responsavel`	`char[50]`	Responsável pela ação (até 49 caracteres).
`participantesPrevistos`	`int`	Quantidade prevista de participantes (0 a 999999).
`participantesEfetivos`	`int`	Quantidade efetiva; só tem valor quando a ação está realizada, e é 0 nos demais casos.
`situacao`	`Situacao`	Planejada, realizada ou cancelada.

Fonte: elaborado pelo autor.

As ações ficam no vetor global `acoes[MAX_ACOES]`. A variável `totalAcoes` indica quantas posições estão em uso: uma nova ação é gravada na posição `totalAcoes`, que só é incrementada depois que todos os dados foram lidos e validados. Assim, um cadastro cancelado não deixa registro incompleto. Cada `Acao` ocupa 240 bytes (valor obtido com `sizeof`), de modo que o vetor completo, com 100 ações, ocupa cerca de 24.000 bytes, reservados desde o início da execução.

4.3 Funções

O programa é dividido em 17 funções, cada uma com uma responsabilidade (Tabela 5). A função main concentra apenas o controle do fluxo principal.

Tabela 5 – Funções do programa e suas responsabilidades

Função	Responsabilidade	Chamada por
`main`	Controla o laço do menu e chama a função escolhida.	—
`exibirMenu`	Mostra o menu, lê e valida a opção (0 a 5); devolve -1 se a entrada for inválida.	`main`
`cadastrarAcao`	Lê os dados de uma ação, valida e grava no vetor.	`main`
`listarAcoes`	Exibe todas as ações em tabela.	`main`
`pesquisarAcoes`	Pesquisa por código, escola ou tema e exibe os resultados.	`main`
`atualizarSituacao`	Altera a situação de uma ação, respeitando as regras de transição.	`main`
`gerarResumo`	Calcula e exibe quantidades, total de participantes e percentual.	`main`
`codigoExiste`	Devolve a posição da ação com o código informado, ou -1 se não existir.	`cadastrarAcao`, `atualizarSituacao`
`contemIgnoreCase`	Verifica se um texto contém outro, sem diferenciar maiúsculas de minúsculas.	`pesquisarAcoes`
`situacaoParaTexto`	Converte o valor da enumeração em texto legível.	`listarAcoes`, `pesquisarAcoes`, `atualizarSituacao`
`lerLinha`	Lê uma linha com `fgets`, remove a quebra de linha e descarta o excesso digitado.	funções de leitura
`lerTexto`	Lê texto obrigatório, recusando vazio, só espaços e texto longo demais.	`cadastrarAcao`, `pesquisarAcoes`
`lerInteiro`	Lê um número inteiro dentro de um intervalo, recusando letras.	várias funções
`lerData`	Lê e valida uma data no formato DD/MM/AAAA.	`cadastrarAcao`
`lerSituacao`	Lê e valida a situação (1, 2 ou 3).	`cadastrarAcao`, `atualizarSituacao`
`anoBissexto`	Informa se um ano é bissexto.	`diasNoMes`
`diasNoMes`	Informa quantos dias tem um mês de um ano.	`lerData`

Fonte: elaborado pelo autor.

4.4 Fluxo de execução

A função `main` exibe o título e entra em um laço `do...while`: a cada volta chama `exibirMenu`, que devolve a opção escolhida, e um `switch` aciona a função correspondente. O laço termina quando a opção é 0. Se a entrada for inválida, `exibirMenu` já mostrou a mensagem de erro e devolve -1, e o laço simplesmente continua. Se a entrada terminar (fim de arquivo), o programa também encerra, em vez de repetir o menu indefinidamente.

Todas as leituras seguem o mesmo padrão: a linha é lida inteira com `fgets` (função `lerLinha`), convertida com `strtol` quando deve ser número e verificada. Esse caminho foi escolhido no lugar de `scanf` porque o `scanf` deixa a quebra de linha no buffer e não detecta entradas como “12abc”, enquanto o `strtol` informa exatamente onde a conversão parou. Quando a linha digitada é maior que o espaço disponível, o excesso é descartado para não contaminar a pergunta seguinte. Cada função de leitura devolve 1 em caso de sucesso e 0 em caso de erro, e a operação em andamento é cancelada com uma mensagem.

De \ Para	Planejada	Realizada	Cancelada
Planejada	Permitida	Permitida (pede os efetivos)	Permitida
Realizada	Permitida (zera os efetivos)	Permitida (pede novo valor)	**Não permitida**
Cancelada	Permitida	**Não permitida**	Permitida

Fonte: elaborado pelo autor.

5 Utilização do programa

As figuras deste capítulo são registros da execução real do programa com **dados fictícios**. O que o usuário digita aparece em amarelo, as mensagens de sucesso em verde e as de erro em vermelho; essas cores servem apenas para facilitar a leitura das figuras, pois o programa exibe texto simples.

5.1 Menu principal

Ao iniciar, o programa mostra o menu da Figura 1. Para escolher uma opção, digite o número correspondente e pressione Enter. Depois de cada operação, o menu é exibido novamente.

5.2 Cadastrar ação

Digite `1` no menu. O programa fará as perguntas da Tabela 7, na ordem em que aparecem. Responda a cada uma e pressione Enter.

Tabela 7 – Dados solicitados no cadastro

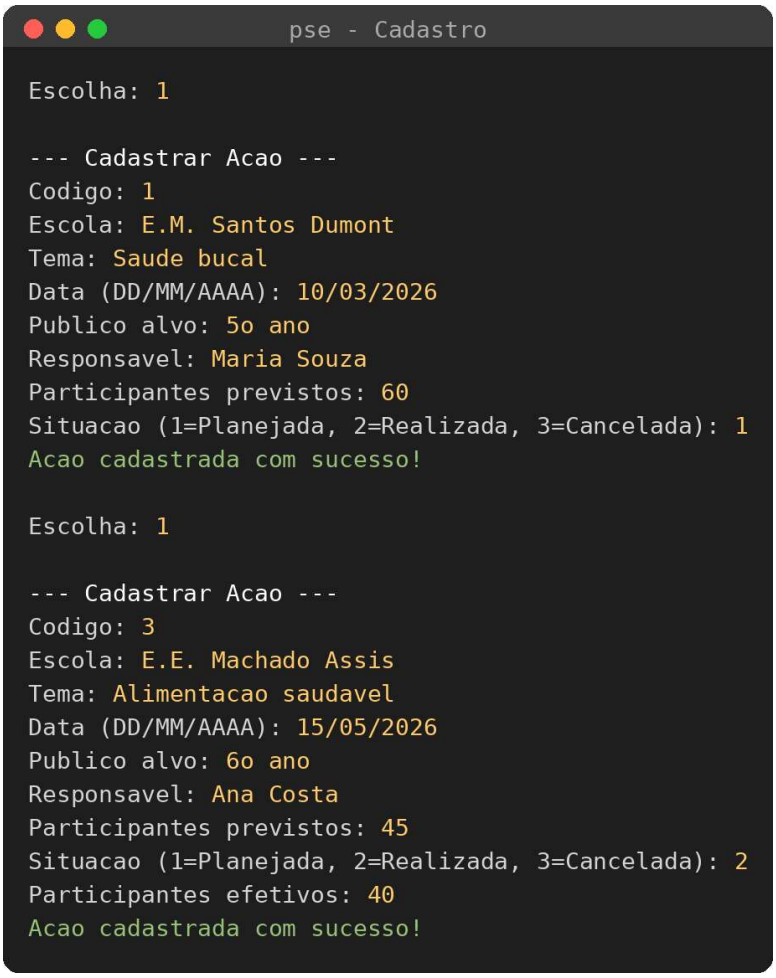
Dado	Formato aceito	Validação
Código	Número de 1 a 999999	Não pode repetir um código já cadastrado.
Escola	Texto de até 59 caracteres	Obrigatório; não aceita só espaços.
Tema	Texto de até 59 caracteres	Obrigatório.

Dado	Formato aceito	Validação
Data prevista	DD/MM/AAAA, com zeros (ex.: 05/03/2026)	Precisa ser uma data real, com controle de meses e anos bissextos.
Público-alvo	Texto de até 39 caracteres	Obrigatório.
Responsável	Texto de até 49 caracteres	Obrigatório.
Participantes previstos	Número de 0 a 999999	Não aceita valores negativos nem letras.
Situação inicial	1 = Planejada, 2 = Realizada, 3 = Cancelada	Somente 1, 2 ou 3. Se for 2, o programa pede os participantes efetivos (0 a 999999).

Fonte: elaborado pelo autor.

A Figura 2 mostra dois cadastros: a ação 1, planejada, e a ação 3, realizada, caso em que o programa pergunta quantos participantes efetivamente compareceram. Ao final, a mensagem “Acao cadastrada com sucesso!” confirma a operação. Se algum dado for inválido, o programa informa o erro e **cancela o cadastro**: nada é gravado e a operação deve ser refeita (veja o Capítulo 6).

Figura 2 – Cadastro de uma ação planejada (código 1) e de uma ação realizada (código 3)



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

5.3 Listar ações

A opção `2` mostra todas as ações em uma tabela com código, escola, tema, data, situação, participantes previstos e participantes efetivos (Figura 3). Para que a tabela caiba em um terminal comum, escola e tema aparecem cortados em 20 caracteres na listagem; os textos completos são mostrados na pesquisa. Se não houver ações cadastradas, o programa informa “Nenhuma acao cadastrada.”.

Figura 3 – Listagem das ações cadastradas

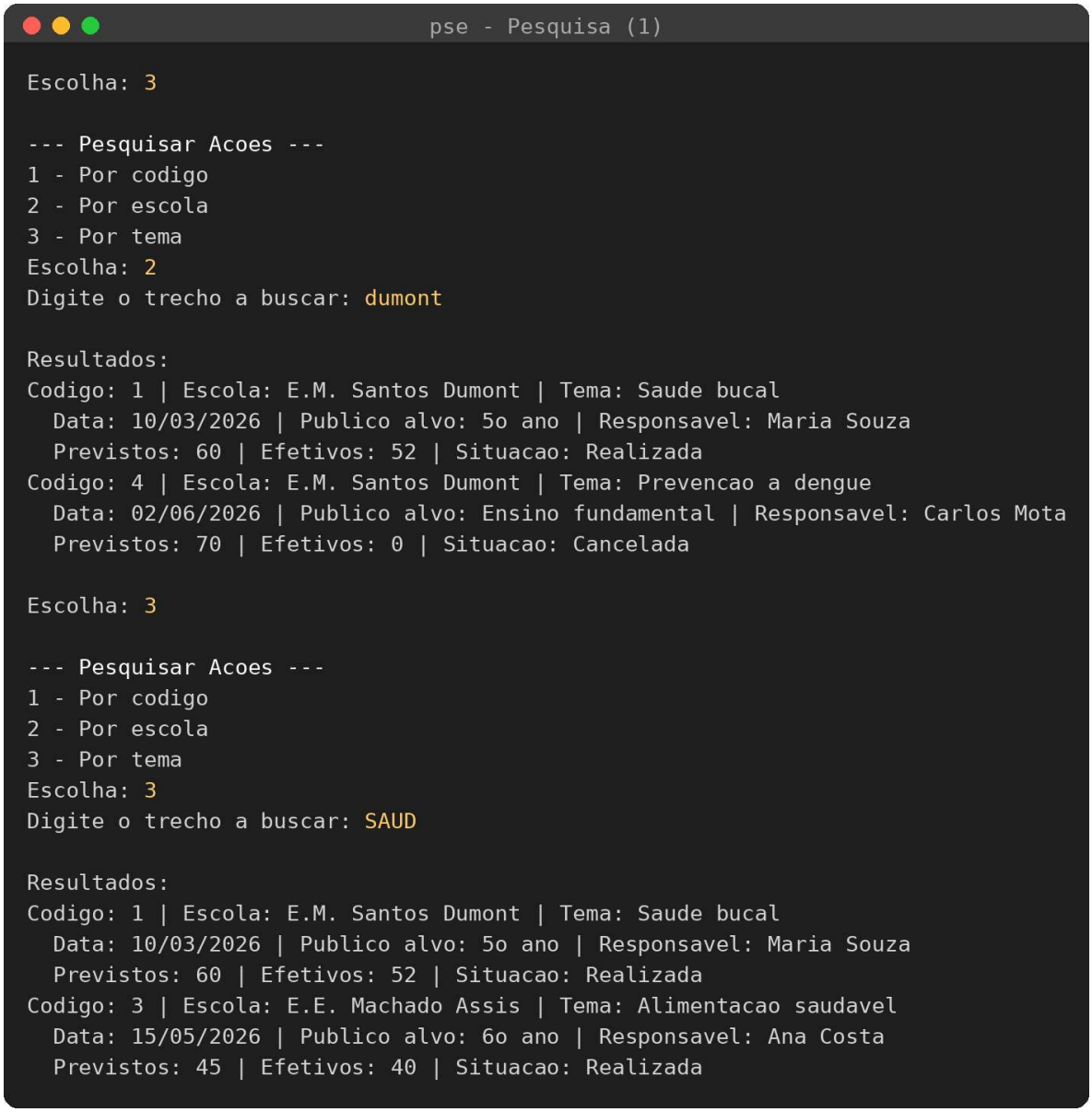


Fonte: saída da execução do programa com dados fictícios, com realce de cores.

5.4 Pesquisar ações

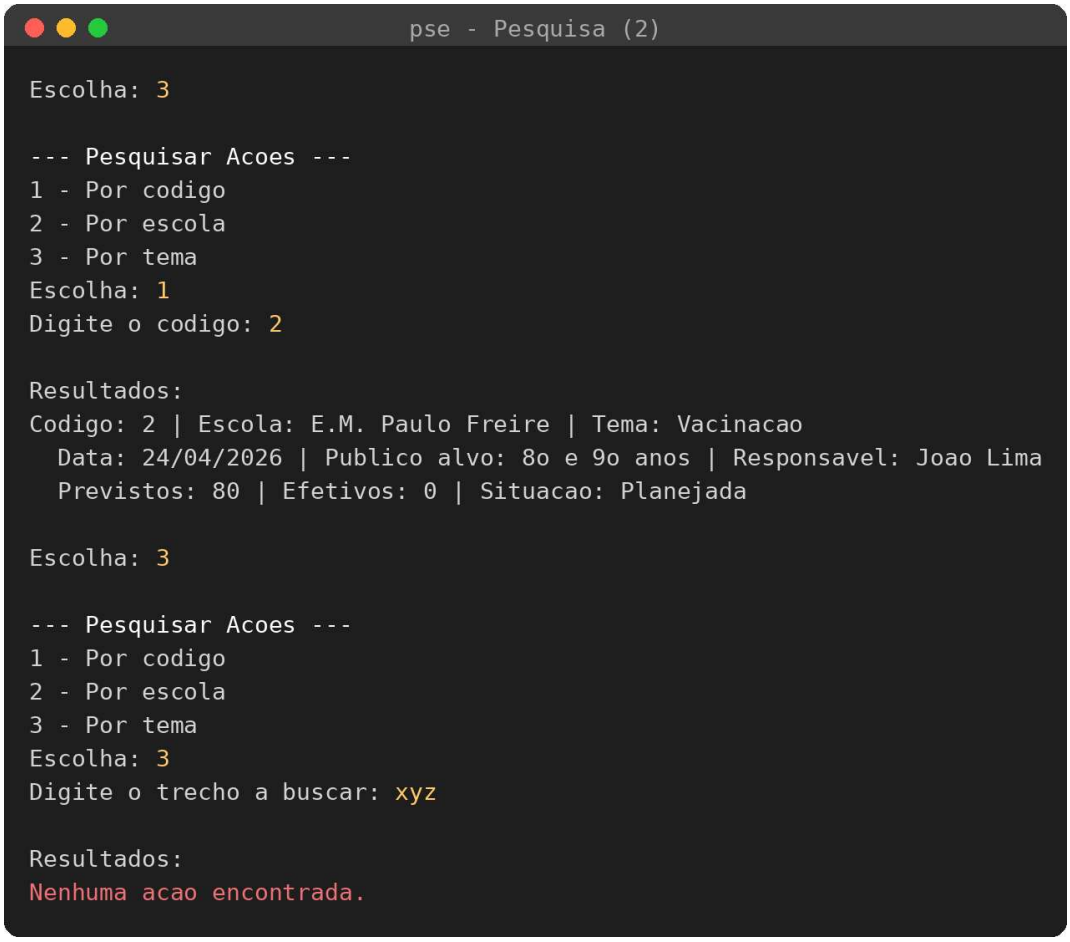
A opção `3` abre um submenu com três critérios: `1` por código, `2` por escola e `3` por tema. A busca por código é exata. As buscas por escola e por tema são **parciais** e **não diferenciam maiúsculas de minúsculas**: ao digitar “SAUD” no tema, o programa encontra “Saude bucal” e “Alimentacao saudavel”. Para cada ação encontrada, são exibidos todos os seus dados. Caso nada seja encontrado, aparece “Nenhuma acao encontrada.”. As Figuras 4 e 5 ilustram os quatro casos.

Figura 4 – Pesquisa por escola (“dumont”) e por tema (“SAUD”)



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

Figura 5 – Pesquisa por código e pesquisa sem resultado



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

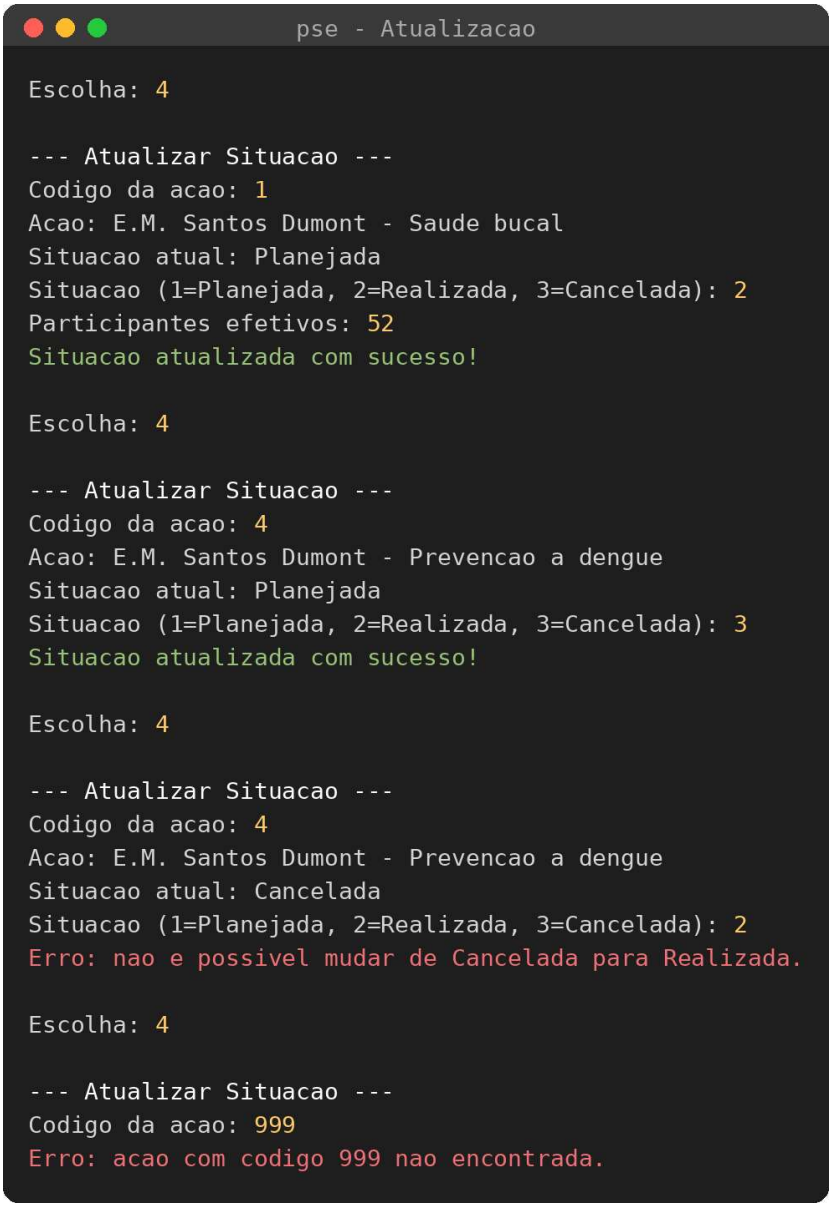
5.5 Atualizar situação

A opção `4` altera a situação de uma ação já cadastrada.

1. Informe o código da ação. O programa mostra a escola, o tema e a situação atual.
2. Escolha a nova situação: 1 (planejada), 2 (realizada) ou 3 (cancelada).
3. Se escolheu 2, informe a quantidade efetiva de participantes.
4. Confirme a mensagem “Situacao atualizada com sucesso!”.

As mudanças proibidas (Tabela 6) são recusadas com uma mensagem explicativa, e um código que não existe também gera erro. A Figura 6 mostra, em sequência: a ação 1 passando a realizada com 52 participantes, a ação 4 sendo cancelada, a tentativa proibida de transformar a ação 4 (cancelada) em realizada e a busca por um código inexistente.

Figura 6 – Atualização de situação: sucesso, mudança proibida e código inexistente

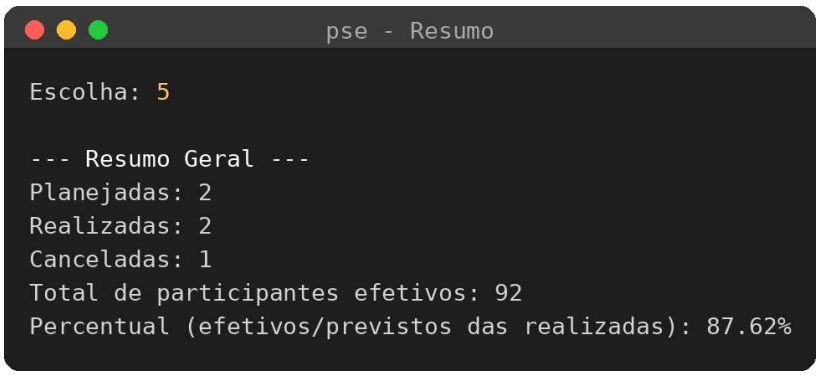


Fonte: saída da execução do programa com dados fictícios, com realce de cores.

5.6 Resumo geral

A opção `5` apresenta a quantidade de ações planejadas, realizadas e canceladas, o total de participantes efetivos e o percentual de participação (Figura 7). Com os dados do exemplo, há 2 ações planejadas, 2 realizadas e 1 cancelada. Os participantes efetivos somam 52 + 40 = 92, e os previstos das ações realizadas somam 60 + 45 = 105, o que resulta em 92 ÷ 105 × 100 = 87,62%. O programa exibe o número com ponto decimal (87.62%).

Figura 7 – Resumo geral das ações



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

5.7 Encerrar o programa

A opção `0` exibe “Saindo...” e encerra o programa. Como os dados ficam apenas na memória, **todas as ações cadastradas são perdidas ao encerrar**.

6 Validações e mensagens de erro

O programa valida as entradas importantes e informa cada problema com uma mensagem clara (Tabela 8). Em todos os casos de erro durante uma operação, nada é gravado e o programa volta ao menu.

Tabela 8 – Validações, mensagens exibidas e efeito

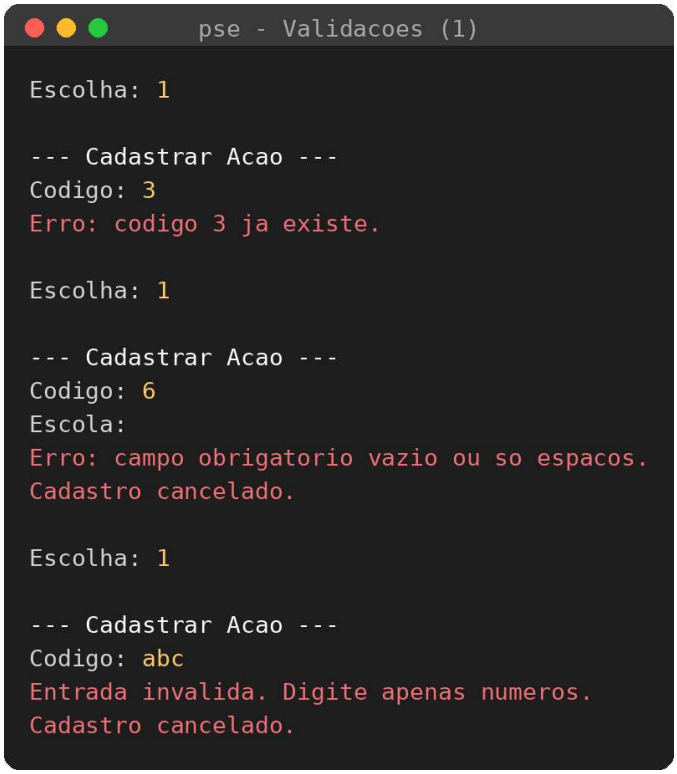
Situação detectada	Mensagem exibida	Efeito
Código repetido	`Erro: codigo N ja existe.`	Cadastro encerrado.
Campo obrigatório vazio ou só com espaços	`Erro: campo obrigatorio vazio ou so espacos.`	Cadastro cancelado.
Texto maior que o limite do campo	`Erro: texto excede o limite de N caracteres.`	Operação cancelada.
Valor não numérico	`Entrada invalida. Digite apenas numeros.`	Operação cancelada.
Número fora do intervalo (inclui negativos)	`Valor fora do intervalo permitido (min a max).`	Operação cancelada.
Data em formato incorreto	`Data invalida. Use DD/MM/AAAA.`	Cadastro cancelado.
Data com letras ou símbolos	`Data invalida. Use apenas numeros em dia, mes e ano.`	Cadastro cancelado.
Data inexistente (ex.: 31/02, 29/02 em ano comum)	`Data invalida. Confira dia, mes e ano.`	Cadastro cancelado.
Opção de menu inexistente	`Opcao invalida. Digite um numero de 0 a 5.`	Volta ao menu.
Vetor cheio (100 ações)	`Erro: vetor cheio (maximo 100 acoes).`	Cadastro recusado.
Código não encontrado na atualização	`Erro: acao com codigo N nao encontrada.`	Operação encerrada.

Situação detectada	Mensagem exibida	Efeito
Mudança de situação proibida	`Erro: nao e possivel mudar de X para Y.`	Situação mantida.
Nenhuma ação cadastrada	`Nenhuma acao cadastrada.`	Aviso em listar, pesquisar e atualizar.
Pesquisa sem resultado	`Nenhuma acao encontrada.`	Aviso.

Fonte: elaborado pelo autor.

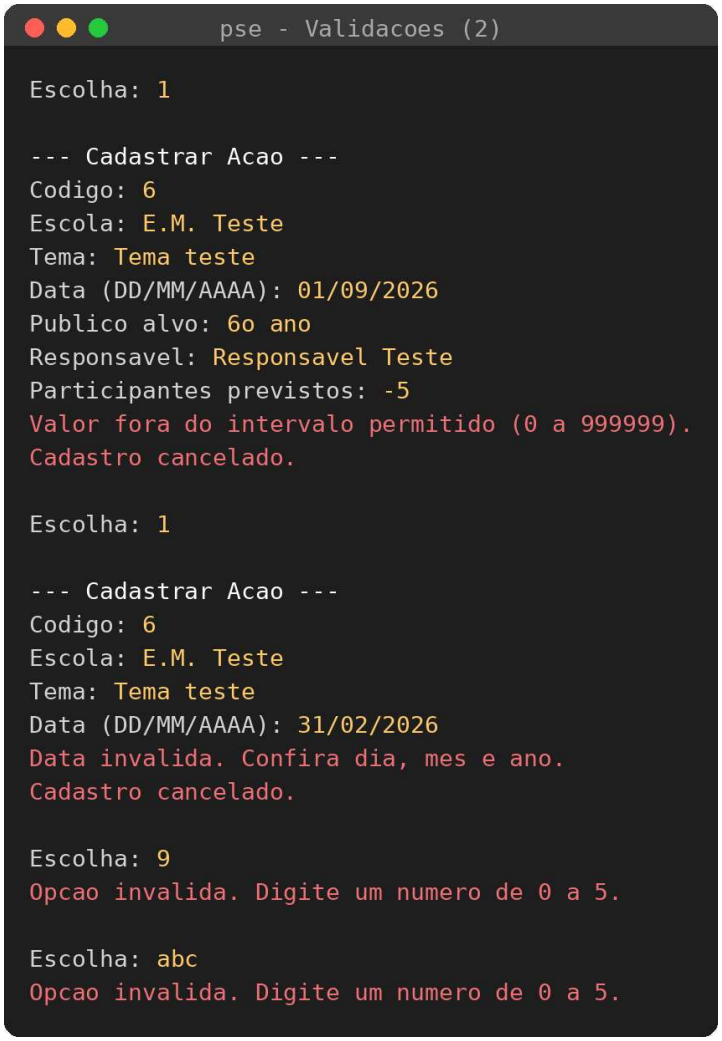
As Figuras 8 e 9 mostram algumas dessas validações em funcionamento.

Figura 8 – Validações no cadastro: código repetido, campo em branco e valor não numérico



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

Figura 9 – Quantidade negativa, data inexistente e opções inválidas no menu



Fonte: saída da execução do programa com dados fictícios, com realce de cores.

7 Testes e resultados

7.1 Metodologia

Os casos de teste da Tabela 9 foram executados com o programa compilado com o GCC 13.3.0 e as opções `-Wall -Wextra -std=c99`. As entradas foram digitadas em um terminal simulado, linha a linha, e o texto produzido pelo programa foi comparado com o resultado esperado. O caso T25 foi executado com um arquivo de entrada contendo 100 cadastros seguidos de uma 101ª tentativa.

7.2 Casos de teste

Tabela 9 – Casos de teste executados e resultados

Nº	Caso	Entrada	Resultado esperado	Resultado
T01	Cadastro válido (situação Planejada)	Código 1, dados válidos, situação 1	`Acao cadastrada com sucesso!`	Aprovado
T02	Cadastro válido como Realizada	Situação 2 e 40 participantes efetivos	Pede os efetivos e cadastra a ação	Aprovado
T03	Código repetido	Código 3 (já cadastrado)	`Erro: codigo 3 ja existe.`	Aprovado
T04	Campo obrigatório só com espaços	Escola = três espaços	Mensagem de erro e cadastro cancelado	Aprovado
T05	Código não numérico	Código = abc	Mensagem de erro e cadastro cancelado	Aprovado
T06	Quantidade negativa	Participantes previstos = -5	`Valor fora do intervalo permitido`	Aprovado
T07	Data inexistente	Data = 31/02/2026	`Data invalida`	Aprovado
T08	Ano não bissexto	Data = 29/02/2027	`Data invalida`	Aprovado
T09	Ano bissexto	Data = 29/02/2028	Cadastro aceito	Aprovado
T10	Opção inexistente no menu (número)	Opção 9	`Opcao invalida`	Aprovado
T11	Opção inexistente no menu (letras)	Opção abc	`Opcao invalida`	Aprovado
T12	Texto com 200 caracteres	Escola com 200 letras “x”	Erro de limite; a entrada seguinte não é afetada	Aprovado
T13	Listar após entrada longa (sem sobra no buffer)	Opção 2 logo após o T12	Lista normal com 1 ação	Aprovado
T14	Pesquisa parcial, sem diferenciar maiúsculas	Tema = SAUD	Ações 3 e 1 (Saude bucal e Alimentacao saudavel)	Aprovado
T15	Pesquisa por escola	Escola = dumont	Ações 1 e 4	Aprovado
T16	Pesquisa por código	Código = 2	Somente a ação 2	Aprovado
T17	Pesquisa sem resultado	Tema = xyz	`Nenhuma acao encontrada.`	Aprovado
T18	Atualizar para Realizada com efetivos	Ação 1, situação 2, efetivos 52	`Situacao atualizada com sucesso!`	Aprovado
T19	Atualizar para Cancelada	Ação 4, situação 3	`Situacao atualizada com sucesso!`	Aprovado
T20	Cancelada não pode virar Realizada	Ação 4, situação 2	`Erro: nao e possivel mudar de Cancelada para Realizada.`	Aprovado
T21	Código inexistente na atualização	Código 999	`Erro: acao com codigo 999 nao encontrada.`	Aprovado
T22	Resumo geral com dados	2 planejadas, 2 realizadas e 1 cancelada	92 efetivos e 87.62%	Aprovado
T23	Resumo sem ações cadastradas	Opção 5 com o vetor vazio	Zeros, sem travar	Aprovado
T24	Listar, pesquisar e atualizar sem ações	Opções 2, 3 e 4 com o vetor vazio	`Nenhuma acao cadastrada.`	Aprovado
T25	Capacidade máxima do vetor	100 cadastros e uma 101ª tentativa	`Erro: vetor cheio (maximo 100 acoes).`	Aprovado

Fonte: elaborado pelo autor, a partir da execução do programa.

7.3 Discussão dos resultados

Os 25 casos produziram o resultado esperado. As validações exigidas no trabalho (código repetido, campos vazios, quantidades negativas e opções inexistentes no menu) funcionaram em todos os testes, e as entradas inválidas nunca chegaram ao vetor de ações. Os casos T12 e T13 mostram um ponto de robustez: ao digitar uma linha de 200 caracteres, o programa recusa o texto e a pergunta seguinte continua normal, porque o excesso digitado é descartado. Sem esse cuidado, o restante da linha seria lido como resposta às perguntas seguintes.

O resumo da Figura 7 permite discutir uma decisão de projeto. O percentual de 87,62% considera apenas as ações realizadas (92 efetivos para 105 previstos). Se todas as ações fossem consideradas, o total previsto seria 305 (60 + 80 + 45 + 70 + 50) e o percentual cairia para 30,16%, valor que não representa a participação real, pois inclui ações que ainda não aconteceram ou que foram canceladas. Por isso, a comparação foi restrita às ações realizadas. Caso o enunciado do trabalho peça outro critério, basta alterar o cálculo em `gerarResumo`.

Quanto aos recursos, o programa reserva cerca de 24.000 bytes para as ações, responde instantaneamente a todas as operações e compila sem nenhum aviso com as opções `-Wall -Wextra`. Os testes de vetor vazio (T23 e T24) confirmam que o programa não trava nem exibe valores incoerentes quando ainda não há dados.

8 Limitações

Algumas limitações do programa decorrem do escopo mínimo do trabalho; outras são pontos que poderiam ser aperfeiçoados:

- **Dados voláteis:** tudo é guardado apenas na memória e se perde ao encerrar o programa. O uso de arquivo era opcional no enunciado.
- **Capacidade fixa:** o vetor comporta no máximo 100 ações.
- **Sem edição ou exclusão:** depois de cadastrada, só é possível alterar a situação da ação; um erro de digitação em outro campo não pode ser corrigido.
- **Cadastro cancelado por inteiro:** qualquer erro de digitação cancela o cadastro e obriga a digitar tudo de novo.
- **Acentuação:** o programa trabalha com texto sem acentos. Com letras acentuadas (que ocupam mais de um byte no terminal em UTF-8), o limite de

caracteres, o alinhamento da tabela e a busca podem se comportar de forma inesperada, e “Saúde” não seria encontrado ao pesquisar “Saude”.

- **Participantes efetivos maiores que os previstos:** o programa aceita esse valor, o que pode levar a percentuais acima de 100%.
- **Datas sem contexto:** a data é validada como data real, mas o programa não verifica se está no passado ou no futuro, nem se combina com a situação (por exemplo, uma ação realizada com data futura).
- **Ações duplicadas:** apenas o código é verificado; a mesma ação pode ser cadastrada duas vezes com códigos diferentes.
- **Listagem simples:** as ações aparecem na ordem de cadastro, sem ordenação, filtro ou paginação, e escola e tema são cortados em 20 caracteres na tabela.
- **Critério do percentual:** o cálculo considera só as ações realizadas, conforme a decisão discutida no item 7.3.
- **Interface:** o uso é apenas por console, sem controle de acesso ou registro de quem fez cada alteração.

9 Melhorias futuras

A Tabela 10 reúne melhorias que corrigem as limitações apontadas ou ampliam o programa, sem alterar seu propósito.

Tabela 10 – Melhorias propostas

Melhoria	Benefício	Como poderia ser feita
Gravar e ler as ações em arquivo	Os dados deixam de se perder ao encerrar.	Funções `fopen`, `fprintf`, `fgets` e `fclose`, com arquivo de texto ou CSV.
Editar e excluir ações	Permite corrigir erros sem recadastrar.	Novas opções no menu, reaproveitando `codigoExiste` e as funções de leitura.
Repetir só o campo inválido	Evita perder os dados já digitados.	Colocar cada leitura em um laço que repete a pergunta até a resposta ser válida.
Alocação dinâmica	Remove o limite de 100 ações.	Usar `malloc` e `realloc` no lugar do vetor de tamanho fixo.
Validar efetivos e previstos	Evita percentuais acima de 100%.	Recusar ou pedir confirmação quando efetivos forem maiores que previstos.
Detectar ações duplicadas	Evita registros repetidos.	Comparar escola, tema e data antes de gravar.
Ordenar e filtrar a listagem	Facilita a leitura com muitas ações.	Ordenar por data, código ou escola; filtrar por situação.

Melhoria	Benefício	Como poderia ser feita
Suporte a acentuação	Permite escrever os textos em português correto.	Configurar a localidade (`setlocale`) e tratar UTF-8.
Resumo por escola ou por tema	Apoia a análise de cada escola.	Somar quantidades e participantes agrupando por escola ou tema.
Dividir o código em vários arquivos	Facilita a manutenção e o reuso.	Arquivos `.c` e `.h` por assunto (leitura, ações, resumo).

Fonte: elaborado pelo autor.

10 Conclusão da implementação

O Gerenciador de Ações do PSE cumpre o escopo mínimo proposto: cadastra, lista, pesquisa e atualiza ações, gera o resumo com o percentual de participação e valida as entradas importantes, tudo em linguagem C padrão, com dados em memória e sem bibliotecas além das padrão. A divisão em funções de responsabilidade clara e o padrão único de leitura e validação tornam o código fácil de entender e de testar, como mostraram os 25 casos executados.

As limitações identificadas, em especial a perda dos dados ao encerrar e a impossibilidade de editar ações, indicam os próximos passos naturais do projeto. Este manual reúne o que é necessário para compilar, usar e evoluir o programa, e as melhorias propostas servem de roteiro para uma próxima versão.

PARTE COMPLEMENTAR — INTEGRAÇÃO DOS ARTEFATOS DA AEP

Esta parte integra os artefatos de planejamento do 1º bimestre ao manual e à especificação do sistema efetivamente desenvolvido no 2º bimestre. O documento IntegraPSE_AEP_ABNT-1.docx é tratado como registro histórico do protótipo, e não como descrição literal das funcionalidades disponíveis na versão final.

11 Evolução do projeto: protótipo e produto final

11.1 Proposta inicial — primeiro bimestre

O protótipo IntegraPSE era direcionado às ações de vacinação do Programa Saúde na Escola. Previa cadastro separado de escolas (código, nome, cidade e estado), vinculação das ações a essas escolas, datas de início e fim, cadastro e exclusão de escolas e ações, registro de alunos vacinados, listagens filtradas por situação e indicadores por escola, além de cálculos de alunos não atendidos. Os artefatos iniciais incluíam 12 requisitos funcionais, 7 não funcionais, fluxogramas e pseudocódigo para esse escopo.

11.2 Implementação — segundo bimestre

O produto implementado em C, identificado no manual como Gerenciador de Ações do PSE, adota uma estrutura simplificada e generalizada: cada ação guarda diretamente o nome da escola, sem cadastro independente de escolas. O sistema trabalha com uma única data prevista, tema, público-alvo, responsável, participantes previstos e efetivos e situação. Dispõe de cadastro, listagem, pesquisa, atualização de situação e resumo geral. Não implementa exclusão, edição geral dos dados, indicadores separados por escola ou cálculo explícito de faltas.

11.3 Quadro comparativo das alterações

Dimensão	Protótipo — 1º bimestre	Produto — 2º bimestre
Foco	Vacinação e alunos vacinados	Ações coletivas do PSE e participantes
Escolas	Cadastro próprio com código/cidade/UF	Nome da escola dentro da ação
Datas	Data inicial e final	Uma data prevista
Operações	CRUD de escolas e ações; filtros específicos	Cadastrar, listar, pesquisar e atualizar situação
Pesquisa	Consultas por escola e situação	Código exato ou texto parcial de escola/tema
Resultados	Vacinados, não atendidos, totais por escola	Participantes efetivos e percentual global das realizadas
Situação	Programada, concluída, cancelada	Planejada, realizada, cancelada

Persistência	Vetor em memória	Vetor em memória com até 100 ações
Privacidade	Sem identificação individual de estudantes	Sem dados clínicos ou individuais

Justificativa documental da revisão: a versão final priorizou a implementação e validação das funcionalidades mínimas previstas para a AEP, em linguagem C e armazenamento em memória. Essa é uma interpretação da diferença entre os arquivos fornecidos; não há registro documental que permita atribuir a decisão a uma reunião ou aprovação específica.

12 Revisão e rastreabilidade dos artefatos

O quadro abaixo evita confundir requisitos propostos na primeira etapa com requisitos atendidos pela versão executável. A numeração RF01–RF08 a seguir pertence à especificação revisada do produto final, não à numeração RF01–RF12 do protótipo.

Artefato original	Situação no produto final	Revisão aplicada
Cadastro de escola independente	Não implementado	Substituído por campo escola no cadastro de ação
Cadastro de ação de vacinação	Adaptado	Cadastro de ação geral com tema e responsável
Consulta de escolas	Não implementada	Pesquisa de ações pelo nome da escola
Consulta e listagem de ações	Implementadas com adaptação	Listagem geral e pesquisa por código/escola/tema
Registro de vacinados	Adaptado	Registro de participantes efetivos nas ações realizadas
Situação programada/concluída/cancelada	Adaptado	Planejada/realizada/cancelada com restrições de transição
Faltas e indicadores por escola	Não implementados	Resumo agregado de ações realizadas e percentual
Exclusão e edição completa	Não implementadas	Somente atualização da situação
Fluxograma geral e detalhado do protótipo	Histórico, requer revisão	Representações atualizadas descritas na seção 14
Pseudocódigo inicial	Histórico, requer revisão	Algoritmo revisado apresentado na seção 15

13 Diagrama e casos de uso do produto final

A figura enviada representa o ator Usuário e os grupos cadastro, consulta, atualização, relatórios e encerramento. Algumas ações internas (validar, localizar, calcular) aparecem como casos auxiliares; elas também são descritas como etapas dos casos principais, evitando confundi-las com funcionalidades autônomas do menu.

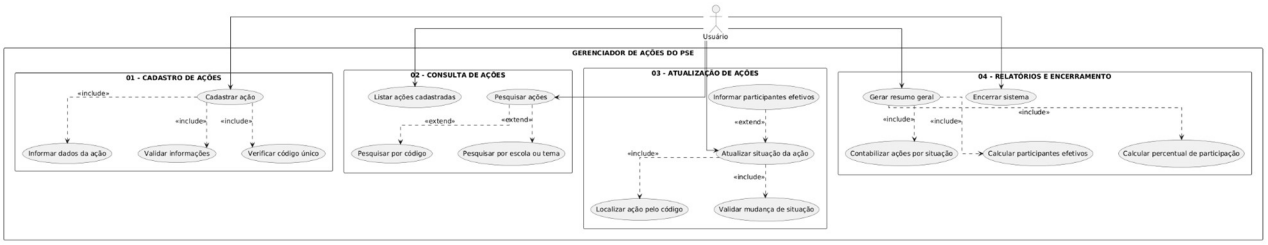


Figura — Diagrama de casos de uso fornecido pelo grupo. Fonte: autores (2026).

CU01 — Cadastrar ação

Ator: Usuário
Pré-condição: Nenhuma; existir espaço no vetor
Fluxo principal: Seleciona 1; informa código, escola, tema, data, público-alvo, responsável, previstos e situação; se realizada, informa efetivos; sistema valida e confirma.
Fluxos alternativos e exceções: Código duplicado, campo vazio, data ou quantidade inválida, vetor cheio: cancela a operação.
Pós-condição: Nova ação armazenada na memória.

CU02 — Listar ações

Ator: Usuário
Pré-condição: Sistema em execução
Fluxo principal: Seleciona 2; sistema percorre o vetor e exibe a tabela.
Fluxos alternativos e exceções: Sem registros: informa ausência de ações.
Pós-condição: Nenhum registro é alterado.

CU03 — Pesquisar ações

Ator: Usuário
Pré-condição: Pelo menos uma ação cadastrada
Fluxo principal: Seleciona 3; escolhe código, escola ou tema; informa critério; sistema apresenta os resultados.
Fluxos alternativos e exceções: Critério inválido cancela pesquisa; nenhuma correspondência gera aviso.
Pós-condição: Registros preservados.

CU04 — Atualizar situação

Ator: Usuário
Pré-condição: Ação previamente cadastrada
Fluxo principal: Seleciona 4; informa código; escolhe situação; se realizada, informa efetivos; sistema valida e confirma.
Fluxos alternativos e exceções: Código inexistente; transição realizada↔cancelada proibida; quantidade inválida: não conclui atualização.
Pós-condição: Situação e efetivos atualizados conforme as validações do sistema.

CU05 — Gerar resumo geral

Ator: Usuário
Pré-condição: Sistema em execução
Fluxo principal: Seleciona 5; sistema conta ações por situação e soma previstos/efetivos das realizadas; calcula percentual.

Fluxos alternativos e exceções: Sem previstos nas realizadas: percentual exibido como zero, evitando divisão por zero.
Pós-condição: Resumo exibido; dados preservados.

CU06 — Encerrar sistema

Ator: Usuário
Pré-condição: Sistema em execução
Fluxo principal: Seleciona 0; sistema exibe mensagem e termina.
Fluxos alternativos e exceções: Fim da entrada também pode encerrar a execução.
Pós-condição: Registros em memória deixam de estar disponíveis.

14 Fluxogramas revisados — descrição textual

Fluxo geral revisado: Início → exibir menu → ler e validar opção → [1 cadastrar | 2 listar | 3 pesquisar | 4 atualizar | 5 resumo | 0 encerrar] → retornar ao menu (exceto 0). Uma opção inválida apresenta erro e retorna ao menu.
Fluxo detalhado de cadastro revisado: verificar limite de 100 ações → ler código → validar faixa e unicidade → ler e validar escola, tema, data, público-alvo, responsável e previstos → ler situação → se realizada, ler efetivos → incrementar totalAcoes e confirmar. Qualquer falha interrompe o cadastro sem incrementar totalAcoes.
Os fluxogramas gráficos da primeira entrega permanecem como artefatos históricos: eles retratam menus e cadastros de escola que não correspondem ao código final. As descrições acima especificam a lógica revisada e devem ser usadas caso sejam redesenhados os fluxogramas gráficos.

15 Pseudocódigo revisado do produto final

```
ALGORITMO GerenciadorPSE
CONSTANTE MAX_ACOES = 100
REGISTRO Acao: codigo, escola, tema, data, publicoAlvo,
  responsavel, previstos, efetivos, situacao
VAR acoes[MAX_ACOES], totalAcoes = 0
INICIO
  REPITA
    mostrarMenu()
    opcao <- lerOpcaoValidada()
    ESCOLHA opcao
      1: cadastrarAcao()
      2: listarAcoes()
      3: pesquisarAcoes()
      4: atualizarSituacao()
      5: gerarResumo()
      0: escrever("Saindo...")
    FIMESCOLHA
  ATE opcao = 0
FIM

CADASTRARACAO:
  SE totalAcoes >= MAX_ACOES: avisarLimite(); RETORNAR
  ler codigo; SE codigo invalido OU repetido: cancelar; RETORNAR
  ler e validar escola, tema, data, publicoAlvo, responsavel
  ler e validar previstos >= 0; ler situacao em {1,2,3}
  efetivos <- 0
  SE situacao = REALIZADA: ler e validar efetivos >= 0
  SE alguma validacao falhar: cancelar; RETORNAR
  acoes[totalAcoes] <- novaAcao
  totalAcoes <- totalAcoes + 1

PESQUISARACOES:
  ler criterio: codigo / escola / tema
```

```
  PARA cada acao:
    SE codigo igual OU escola/tema contem trecho sem distinguir caixa:
      exibir dados completos
  SE nenhum resultado: informar

ATUALIZARSITUACAO:
  ler codigo; localizar acao; SE inexistente: informar; RETORNAR
  ler novaSituacao
  SE transicao REALIZADA <-> CANCELADA: rejeitar; RETORNAR
  SE novaSituacao = REALIZADA: ler efetivos validos
  SENAO: efetivos <- 0
  salvar situacao e efetivos

GERARRESUMO:
  contar planejadas, realizadas e canceladas
  somar efetivos e previstos SOMENTE das realizadas
  SE previstos > 0: percentual <- 100 * efetivos / previstos
  SENAO: percentual <- 0
  exibir contagens, efetivos e percentual
```

16 Considerações sobre o fechamento da AEP

O manual anterior contém a especificação funcional revisada, procedimentos de compilação, imagens de uso e tabela de 25 testes. A integração evidencia a diferença entre o planejamento do primeiro bimestre e o produto efetivamente entregue no segundo. Recomenda-se atualizar o sumário automático no Word e conferir o nome final do sistema nos links do repositório e do vídeo antes da submissão.

17 Links dos entregáveis e instruções de acesso

Esta seção reúne os endereços eletrônicos necessários à avaliação da segunda entrega. Os links devem ser preenchidos pelo grupo antes do envio, verificados em uma janela anônima e mantidos acessíveis durante o período de avaliação.

17.1 Repositório do código-fonte (GitHub)

Inserir o endereço público do repositório que contém o código-fonte em C, o arquivo README e as instruções para compilação e execução.

Entregável	
GitHub	https://github.com/sakamotobergamini/Entrega2_AEP.git

17.2 Vídeo de apresentação (YouTube)

Inserir o endereço do vídeo de demonstração do sistema, com participação dos integrantes. Conferir o limite de duração previsto na AEP e a permissão de acesso (público ou não listado).

Entregável	
YouTube	https://www.youtube.com/watch?v=mlky71MCEE4

Conferência final: testar os dois endereços, confirmar que o GitHub contém a versão final do programa e verificar se o vídeo pode ser reproduzido sem solicitação de acesso.